

Blender Expert System v2

Жёсткое техническое задание для Claude Code

Цель: превратить существующего Telegram-бота в бесплатную экспертную систему по Blender без обязательного внешнего AI API. Система должна опираться на качественную локальную базу знаний, версиюность, поиск, диагностику, обучение и автоматические тесты.

Важно: этот документ — полная версия ТЗ, а не сокращённый чек-лист. Требования ниже являются частью единого задания и должны реализовываться поэтапно.

1. Главная цель

- Переделать существующего Telegram-бота в бесплатную экспертную систему по Blender, работающую без OpenAI API, Anthropic API, Gemini API и других платных LLM API.
- Система должна искать информацию в нескольких слоях базы знаний, учитывать версию Blender, ранжировать источники по авторитетности, диагностировать типовые проблемы, вести многошаговый диалог, обучать пользователя, хранить прогресс и честно сообщать, когда данных недостаточно.
- Не пытаться имитировать ChatGPT. Цель — сделать специализированную систему, которая очень хорошо отвечает именно на Blender-вопросы.

2. Жёсткие ограничения

- Основная версия обязана работать без внешней LLM.
- Допустимы Python, SQLite, JSON, YAML, scikit-learn, стандартные Python-библиотеки, локальный full-text search, TF-IDF, cosine similarity, BM25 при необходимости, Telegram Bot API и Git.
- Локальная LLM может быть добавлена позже как необязательный слой. Нельзя делать её фундаментальной зависимостью.

3. Приоритет источников

- S-TIER — 100: официальный Blender Manual, Blender Python API, Blender Release Notes, Blender Developer Documentation и официальные технические документы.
- A-TIER — 80: Blender Stack Exchange, Stack Overflow RU, официальные developer-материалы и качественные технические проекты.
- B-TIER — 60: качественные community tutorials и форумы.
- C-TIER — 30: обычные пользовательские материалы.
- D-TIER — 10: непроверенные статьи, SEO-контент и материалы без автора или версии. D-TIER нельзя использовать как authoritative answer.

4. Обязательные источники знаний

- Индексировать: Blender Manual 5.1 RU; Blender Manual 5.1 EN; Blender Python API 5.1; Blender 5.1 Release Notes; Blender Developer Documentation; Blender Developer Handbook; Blender source code; официальные переводы.
- Исторические официальные версии: 5.0, 4.5, 4.4, 4.3, 4.2, 4.1, 4.0, 3.6. Каждая версия должна иметь собственные metadata.
- Community: Stack Overflow RU / blender, Blender Stack Exchange, forumBlender RU.
- Не загружать массово SEO-контент, AI-generated tutorials, материалы неизвестного происхождения и источники с сомнительной лицензией.

5. Структура knowledge base

- Создать структуру: knowledge/official/manual/5.1; knowledge/official/python_api/5.1; knowledge/official/release_notes; knowledge/official/developer_docs; knowledge/community/stackoverflow_ru; knowledge/community/blender_stackexchange; knowledge/community/forumb Blender; knowledge/personal/dima_notes; knowledge/system/synonyms; knowledge/system/terminology; knowledge/system/intents; knowledge/system/diagnostics; knowledge/system/rules.
- Личные конспекты пользователя хранить отдельно от официальной документации. Бот должен уметь отличать «официально Blender говорит X» от «в личных заметках рекомендуется Y».

6. Metadata каждого knowledge chunk

- Обязательные поля: id, source, source_type, authority, version, language, topic, subtopic, date, url, original_title, translated_title, content.
- Дополнительно желательно хранить: license, author, content_hash, ingested_at, last_updated, parent_document, section_path.

7. Версионный engine

- Поддерживать Blender 5.1.1, 5.1.2, 5.1 и другие версии.
- Если версия неизвестна, не утверждать, что ответ относится к конкретной версии.
- При конфликте версий явно показывать различия.
- Никогда не смешивать старую и новую документацию без явной маркировки.

8. Двязычная терминология

- Для каждого важного Blender term хранить English term, Russian term, aliases и UI label.
- Не удалять английские названия. Поиск должен понимать Mirror, Bevel, Apply Scale, Subdivision и т. п. независимо от языка вопроса.

9. Terminology Database

- Категории: modeling, mesh, topology, modifiers, materials, shaders, rendering, lighting, camera, animation, rigging, geometry nodes, physics, compositing, VSE, UV, texturing, sculpting, Grease Pencil, Python, addons, assets, color management, motion tracking.
- Для каждого термина хранить canonical_name, russian_name, aliases, english_aliases, category, related_terms, common_mistakes.

10. Search Engine

- Не использовать простой поиск по подстроке как основной механизм.
- Комбинировать: exact term match → normalized text match → alias match → TF-IDF similarity → optional BM25 → metadata filtering.
- Финальный score должен учитывать lexical_score, semantic_similarity, authority_score, version_score, topic_score и exact_term_bonus.
- Повышать score при совпадении версии, официального источника, точного термина, категории и типа проблемы. Понижать при старой версии, отсутствии версии и низком authority.

11. Intent Engine

- Без LLM реализовать intents: WHAT_IS, HOW_TO, WHY, ERROR, TROUBLESHOOTING, COMPARISON, WORKFLOW, BEST_PRACTICE, LEARNING, EXAM, TERMINOLOGY, VERSION, PYTHON, GEOMETRY_NODES,

MODELING, MATERIALS, LIGHTING, RENDERING, ANIMATION, RIGGING, COMPOSITING, VSE, UV, SCULPT, EXPORT, IMPORT, ADDON, PERFORMANCE.

- Intent определяется комбинацией keywords, aliases, question patterns, контекста и предыдущих сообщений.

12. Diagnostic Engine

- Создать отдельную базу knowledge/system/diagnostics.
- Каждая проблема: problem_id, title, symptoms, keywords, possible_causes, questions, decision_tree, solutions, version, sources, severity.
- Бот не должен сразу выдавать длинный список причин. При недостатке данных он задаёт один наиболее информативный вопрос и продолжает диагностику по decision tree.

13. Пример диагностического диалога

- Пользователь: «После Subdivision модель ломается».
- Бот: «Где именно появляется проблема? 1) На углах 2) На отверстиях 3) На плоских поверхностях 4) По всей модели».
- Следующий вопрос должен зависеть от ответа. Цель — постепенно исключать причины, а не вываливать все возможные решения сразу.

14. Confidence Engine

- Каждый ответ получает HIGH, MEDIUM, LOW или UNKNOWN.
- HIGH: официальный источник, совпадение версии, точный термин, однозначный факт.
- MEDIUM: несколько качественных community sources или косвенное подтверждение.
- LOW: старый источник, отсутствует версия или проблема неоднозначна.
- UNKNOWN: данных недостаточно.
- LOW нельзя превращать в уверенное утверждение.

15. Source citations

- Технический ответ должен по возможности показывать источник, раздел, версию и URL.
- Community source должен быть явно помечен как community answer.

16. Fact vs Recommendation

- Различать FACT, RULE, RECOMMENDATION, WORKFLOW, OPINION и COMMUNITY PRACTICE.
- Workflow preference нельзя выдавать за правило Blender.
- Формулировки вроде «всегда» и «никогда» использовать только для реально детерминированного поведения, а не для workflow-рекомендаций.

17. Conflict Engine

- Приоритет: Official > Developer > A-tier > B-tier > C-tier.
- Конфликт не удалять. Показывать различия и объяснять, почему выбран источник более высокого приоритета.

18. Education Engine

- Команды: /learn, /progress, /test, /exam, /weaknesses, /next.

- Структура темы: Theory → Example → Exercise → Quiz → Result → Weakness tracking.
- Упражнения должны быть практическими и соответствовать уровню пользователя.

19. User Profile

- SQLite: user_id, blender_version, level, topics, completed_topics, weak_topics, test_results, mistakes, last_questions, learning_goal.
- Хранить только данные, необходимые для работы системы.

20. Level System

- Уровни: Beginner, Junior, Junior+, Middle, Middle+, Senior.
- Не назначать уровень произвольно. Создать competency matrix по Modeling, Topology, Materials, Lighting, Animation, Rendering, Geometry Nodes, Compositing, Motion Design и Python.
- Оценка должна опираться на результаты тестов и практических сценариев.

21. Adaptive Learning

- Ошибки пользователя сохранять.
- При повторяющейся ошибке увеличивать weakness score.
- Следующая рекомендация должна учитывать реальные слабые места, а не просто выдавать случайную тему.

22. Exam Engine

- Типы вопросов: multiple choice, true/false, scenario, diagnostic, workflow, technical.
- После ответа показывать результат, правильный ответ, объяснение, источник, слабое место и следующий шаг.

23. Error Database

- Отдельная база: error_message, normalized_error, cause, solution, version, source, related_errors.
- Поддерживать Python errors, rendering, missing textures, modifiers, geometry nodes, export/import, crashes, viewport, GPU и color management.

24. Язык пользователя

- Основной язык — русский.
- Поиск должен понимать русский, английский, смешанный русский/английский, Blender UI terms и shortcuts.
- Синонимы и пользовательские формулировки должны связываться с canonical technical concept.

25. Response Format

- Simple reference: короткий ответ.
- How-to: цель → шаги → проверка результата → распространённые ошибки → источник.
- Troubleshooting: симптом → наиболее вероятная причина → проверка → решение → если не помогло → источник.
- Learning: что изучаем → почему → что знать → упражнение → проверка → следующий шаг.

26. Zero-Hallucination Mode

- Если информации нет: «Я не нашёл достаточно надёжной информации в базе знаний».
- Не выдумывать техническое объяснение из догадки.
- Если источники противоречат друг другу и система не может разрешить конфликт, честно сообщать об этом.

27. Knowledge Ingestion Pipeline

- SOURCE → FETCH → PARSE → CLEAN → LANGUAGE DETECTION → VERSION DETECTION → TITLE EXTRACTION → TERM EXTRACTION → CHUNKING → METADATA → QUALITY SCORE → INDEX.
- Каждый ingestion должен быть повторяемым и идиempотентным.

28. Structural Chunking

- Не резать документы тупо по количеству символов.
- Учитывать Heading, Subheading, Paragraph, List, Table, Example, Warning.
- Не разрывать code blocks, tables, definitions и procedures.

29. Duplicate Detection

- Обнаруживать одинаковые страницы, переводы, копипасты, старые версии и дубли community articles.
- Не хранить одинаковый текст десятки раз.

30. Source Versioning

- Хранить source_version, ingested_at, content_hash, last_updated.
- При обновлении сравнивать hash и переиндексировать только изменённые chunks.

31. Offline Knowledge

- Во время работы бот должен использовать локальную knowledge base.
- Рабочая схема: Telegram → Local Knowledge.
- Интернет используется для обновления базы, а не как неконтролируемый источник ответа.

32. Admin Commands

- /admin, /reindex, /stats, /sources, /version, /health, /search, /debug.
- Все административные команды доступны только OWNER_ID.

33. Debug Mode

- Показывать intent, detected terms, version, search results, scores, selected source и confidence.
- В production пользователю debug-данные не показывать.

34. Test Suite

- Минимум 400 тестовых случаев: 100 basic, 100 technical, 100 troubleshooting, 50 version, 50 terminology, 50 deliberately ambiguous и 50 вопросов, ответа на которые нет в базе.
- Каждый тест: input, expected_intent, expected_topic, expected_source_tier, expected_answer_elements, expected_confidence.

35. Quality Score

- Метрики: Intent accuracy, Search accuracy, Version accuracy, Source accuracy, Diagnostic accuracy, Citation accuracy и Unknown handling.
- Бес: 20% intent, 20% retrieval, 20% source authority, 15% version, 15% diagnosis, 10% resistance to hallucination.
- Цели: $v2 \geq 70\%$, $v2.5 \geq 80\%$, $v3 \geq 90\%$.

36. Не гнаться за количеством

- 1000 качественных chunks лучше 100000 мусорных chunks.
- Приоритет: точность, актуальность, authority и структура знаний.

37. Knowledge Graph

- Начать с SQLite, а не с тяжёлой графовой БД.
- Связывать сущности: Mirror → Modifier → Clipping/Merge → Origin/Transform/Apply Scale/Topology → типовые проблемы.
- Использовать связи для поиска по смыслу, даже если пользователь не употребил точный термин из документации.

38. Human-language Dictionary

- Создать словарь пользовательских формулировок: «модель как мыло», «как пластилин», «после сглаживания всё поплыло», «рендер чёрный», «текстура пропала», «материал не виден», «ноды ничего не делают» и т. п.
- Каждую формулировку связывать с техническими концепциями и диагностическими деревьями.

39. Архитектура проекта

- Рекомендуемая структура: app/, bot/, knowledge/, search/, intents/, diagnostics/, education/, profile/, tests/, scripts/, config/, data/.
- Не смешивать Telegram-логику, поиск знаний, диагностику и хранилище в одном файле.

40. Порядок реализации

- Phase 1: анализ существующего repository и architecture map.
- Phase 2: refactor architecture.
- Phase 3: knowledge registry.
- Phase 4: официальный Blender Manual.
- Phase 5: version engine.
- Phase 6: terminology.
- Phase 7: search engine.
- Phase 8: intent engine.
- Phase 9: diagnostic engine.
- Phase 10: source ranking.
- Phase 11: education engine.
- Phase 12: user profile.
- Phase 13: test suite.
- Phase 14: optimization.

- Phase 15: production deployment.

41. Правила работы Claude Code

- Перед изменением кода: изучить repository, построить architecture map, определить reusable components и deprecated components.
- Не переписывать проект целиком без необходимости.
- Не удалять работающий функционал без доказанной причины.
- Делать изменения небольшими этапами.
- После каждого этапа запускать tests.
- Не утверждать, что функция работает, пока она реально не протестирована.

42. Отчёт после каждой фазы

- Claude Code обязан сообщать: Changed, Added, Removed, Tests, Known issues, Next phase.
- Если тест провален — указать конкретную причину и не маскировать её формулировкой «почти готово».

43. Финальные критерии готовности v2

- Работает без платного AI API.
- Индексирует официальный Manual.
- Различает версии Blender.
- Понимает русские и английские термины.
- Использует aliases.
- Имеет TF-IDF/BM25 или эквивалентный локальный retrieval.
- Ранжирует источники.
- Показывает источники.
- Умеет диагностировать типовые проблемы.
- Умеет вести multi-turn troubleshooting.
- Имеет UNKNOWN state.
- Имеет SQLite user profile.
- Имеет learning system.
- Имеет automated tests.
- Имеет admin/debug mode.

44. Главный принцип

- Не пытаться симулировать ChatGPT.
- Система должна превосходить обычный FAQ-бот за счёт QUALITY OF KNOWLEDGE + SOURCE AUTHORITY + VERSION AWARENESS + SEARCH + DIAGNOSTIC LOGIC + USER PROFILE + EDUCATION + TESTING.
- Если позже появится локальная LLM, она должна быть дополнительным слоем, а не фундаментом.

Приложение А — рекомендуемые источники

Источник	Tier	Роль
Blender Manual 5.1 RU	S	Основная русскоязычная база
Blender Manual 5.1 EN	S	Проверка и резерв
Blender Python API 5.1	S	Python/API
Blender 5.1 Release Notes	S	Изменения версий
Blender Developer Documentation	S	Разработка и внутренняя логика
Blender GitHub	S	Исходный код
Blender Stack Exchange	A	Реальные технические проблемы
Stack Overflow RU / Blender	A	Русскоязычные вопросы
forumBlender RU	B	Практический community-контент
Личные заметки пользователя	Custom	Отдельный personal tier

Примечание: PDF пересобран с встраиваемым Unicode-шрифтом с поддержкой кириллицы. Предыдущая версия использовала стандартный Helvetica, который не является подходящим шрифтом для полноценного русского текста в ReportLab; документация ReportLab отдельно указывает, что для Unicode необходимо использовать шрифт, содержащий нужные glyphs. `[cite]turn0search0`